CORRIGÉ COMPLET

Application de gestion d’une agence de voyages — UML et C++

Objectif : analyser le sujet, construire le diagramme de classes UML et réaliser une application console en C++ qui applique les règles de gestion.

À savoir : cette version est un projet pédagogique en C++17. Les données sont gardées en mémoire pendant l’exécution ; elles ne sont pas sauvegardées après la fermeture.

1. Analyse du sujet

Une agence de voyages souhaite informatiser la gestion de ses billets d’avion. L’application doit gérer les passagers, les vols, les réservations, les paiements et les annulations. L’utilisateur doit pouvoir enregistrer un passager, ajouter et rechercher un vol, réserver, payer, annuler et consulter les réservations.

2. Règles de gestion

N°	Règle	Application dans le programme
1	Le passager doit être enregistré avant de réserver.	Vérification de l’identifiant du passager.
2	Un vol ne peut pas dépasser sa capacité.	Calcul des places restantes avant réservation.
3	Un passager ne réserve pas deux fois le même vol.	Recherche d’une réservation active identique.
4	Une réservation concerne un seul passager et un seul vol.	Chaque réservation mémorise les deux identifiants.
5	Une réservation annulée ne peut plus être utilisée.	Son statut devient « Annulée ».
6	Le paiement est associé à une réservation.	Le paiement mémorise l’identifiant de réservation.
7	Un paiement déjà remboursé ne doit pas l’être une seconde fois.	Le statut du paiement est contrôlé.

3. Diagramme de classes UML

Chaque classe est un rectangle divisé en trois parties : son nom, ses attributs (données) et ses méthodes (actions). Le signe « - » signifie privé ; le signe « + » signifie public.



Lecture des associations

Passager–Réservation : un passager peut avoir 0 à plusieurs réservations (0..*) ; chaque réservation appartient à un seul passager (1).

Vol–Réservation : un vol peut recevoir 0 à plusieurs réservations ; chaque réservation concerne un seul vol.
Réservation–Paiement : une réservation peut avoir 0 ou 1 paiement dans ce modèle simplifié ; chaque paiement concerne une réservation.

Multiplicités : 1 = exactement un ; 0..1 = zéro ou un ; 0..* = zéro ou plusieurs ; 1..* = un ou plusieurs.

4. Explication des classes

Classe	Attributs	Rôle et méthodes
Passager	idPassager:int, nom:string, prenom:string, telephone:string, email:string	Représente un client. Méthodes : enregistrer, rechercher, afficher.
Vol	numeroVol:string, villeDepart:string, villeArrivee:string, dateDepart:string, capacite:int, prix:double	Représente un vol. Méthodes : ajouter, vérifier les places, afficher.
Reservation	idReservation:int, dateReservation:string, statut:string, idPassager:int, numeroVol:string	Relie un passager à un vol. Méthodes : réserver, valider, annuler, vérifier les doublons.
Paiement	idPaiement:int, idReservation:int, montant:double, datePaiement:string, statutPaiement:string	Gère le paiement et le remboursement d’une réservation.

5. Déroulement d’une réservation

1. Enregistrer le passager.
2. Ajouter le vol.
3. Rechercher le vol et vérifier les places disponibles.
4. Vérifier que le passager n’a pas déjà réservé ce vol.
5. Créer la réservation avec le statut « En attente ».
6. Enregistrer le paiement : la réservation devient « Confirmée ».
7. En cas d’annulation, la place est libérée et le paiement est marqué « Remboursé » s’il avait été payé.

6. Code source complet en C++17

Copie le code dans un fichier main.cpp. Il utilise les classes, l’encapsulation, les constructeurs, les vecteurs et un menu interactif.

```
#include <iostream>
#include <vector>
#include <string>
#include <limits>
using namespace std;

class Passager {
private:
    int id;
    string nom, prenom, telephone, email;
public:
    Passager(int i, string n, string p, string t, string e)
        : id(i), nom(n), prenom(p), telephone(t), email(e) {}
    int getId() const { return id; }
    void afficher() const {
        cout << id << " | " << prenom << " " << nom
             << " | " << telephone << " | " << email << '\n';
    }
};

class Vol {
private:
    string numero, depart, arrivee, date;
    int capacite;
    double prix;
public:
    Vol(string n, string d, string a, string dt, int c, double p)
        : numero(n), depart(d), arrivee(a), date(dt), capacite(c), prix(p) {}
    string getNumero() const { return numero; }
    string getDepart() const { return depart; }
    string getArrivee() const { return arrivee; }
    int getCapacite() const { return capacite; }
    double getPrix() const { return prix; }
    void afficher(int places) const {
        cout << numero << " | " << depart << " -> " << arrivee
             << " | " << date << " | " << prix << " FCFA | Places: "
             << places << "/" << capacite << '\n';
    }
};

class Reservation {
private:
    int id, idPassager;
    string numeroVol, date, statut;
public:
    Reservation(int i, int p, string v, string d)
        : id(i), idPassager(p), numeroVol(v), date(d), statut("En attente") {}
    int getId() const { return id; }
    int getIdPassager() const { return idPassager; }
    string getNumeroVol() const { return numeroVol; }
    string getStatut() const { return statut; }
    bool active() const { return statut == "En attente" || statut == "Confirmee"; }
    void confirmer() { statut = "Confirmee"; }
    void annuler() { statut = "Annulee"; }
    void afficher() const {
        cout << "Reservation #" << id << " | Passager #" << idPassager
             << " | Vol " << numeroVol << " | " << date
             << " | " << statut << '\n';
    }
};

class Paiement {
private:
    int id, idReservation;
    double montant;
    string date, statut;
public:
    Paiement(int i, int r, double m, string d)
        : id(i), idReservation(r), montant(m), date(d), statut("Paye") {}
    int getIdReservation() const { return idReservation; }
    string getStatut() const { return statut; }
    void rembourser() { if (statut == "Paye") statut = "Rembourse"; }
    void afficher() const {
        cout << "Paiement #" << id << " | Reservation #" << idReservation
             << " | " << montant << " FCFA | " << date << " | " << statut << '\n';
    }
};

class AgenceVoyage {
private:
    vector<Passager> passagers;
    vector<Vol> vols;
    vector<Reservation> reservations;
    vector<Paiement> paiements;
    int idP = 1, idR = 1, idPay = 1;

    int trouverPassager(int id) const {
        for (int i=0; i<(int)passagers.size(); ++i)
            if (passagers[i].getId() == id) return i;
        return -1;
    }
    int trouverVol(const string& n) const {
        for (int i=0; i<(int)vols.size(); ++i)
            if (vols[i].getNumero() == n) return i;
        return -1;
    }
    int trouverReservation(int id) const {
        for (int i=0; i<(int)reservations.size(); ++i)
            if (reservations[i].getId() == id) return i;
        return -1;
    }
    int trouverPaiement(int idR) const {
        for (int i=0; i<(int)paiements.size(); ++i)
            if (paiements[i].getIdReservation() == idR) return i;
        return -1;
```

```
    }
    int placesRestantes(const string& n) const {
        int iv = trouverVol(n);
        if (iv == -1) return 0;
        int occupees = 0;
        for (const auto& r : reservations)
            if (r.getNumeroVol() == n && r.active()) ++occupees;
        return vols[iv].getCapacite() - occupees;
    }
    void viderLigne() {
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

public:
    void ajouterPassager() {
        string n,p,t,e;
        cout << "Nom : "; getline(cin,n);
        cout << "Prenom : "; getline(cin,p);
        cout << "Telephone : "; getline(cin,t);
        cout << "Email : "; getline(cin,e);
        passagers.emplace_back(idP++,n,p,t,e);
        cout << "Passager enregistre.\n";
    }

    void ajouterVol() {
        string n,d,a,dt; int c; double p;
        cout << "Numero du vol : "; getline(cin,n);
        if (trouverVol(n) != -1) { cout << "Numero deja utilise.\n"; return; }
        cout << "Ville de depart : "; getline(cin,d);
        cout << "Ville d'arrivee : "; getline(cin,a);
        cout << "Date (JJ/MM/AAAA) : "; getline(cin,dt);
        cout << "Capacite : "; if (!(cin>>c)) { cin.clear(); viderLigne(); return; }
        cout << "Prix en FCFA : "; if (!(cin>>p)) { cin.clear(); viderLigne(); return; }
        viderLigne();
        if (c <= 0 || p < 0) { cout << "Valeurs invalides.\n"; return; }
        vols.emplace_back(n,d,a,dt,c,p);
        cout << "Vol ajoute.\n";
    }

    void afficherPassagers() const {
        if (passagers.empty()) cout << "Aucun passager.\n";
        for (const auto& p : passagers) p.afficher();
    }

    void afficherVols() const {
        if (vols.empty()) cout << "Aucun vol.\n";
        for (const auto& v : vols) v.afficher(placesRestantes(v.getNumero()));
    }

    void rechercherVol() const {
        string d,a;
        cout << "Ville de depart : "; getline(cin,d);
        cout << "Ville d'arrivee : "; getline(cin,a);
        bool trouve = false;
        for (const auto& v : vols) {
            if (v.getDepart() == d && v.getArrivee() == a) {
                v.afficher(placesRestantes(v.getNumero()));
                trouve = true;
            }
        }
        if (!trouve) cout << "Aucun vol pour ce trajet.\n";
    }

    void reserver() {
        int id; string n,dt;
        cout << "ID du passager : ";
        if (!(cin>>id)) { cin.clear(); viderLigne(); return; }
        viderLigne();
        if (trouverPassager(id) == -1) {
            cout << "Refus : passager non enregistre.\n"; return;
        }
        cout << "Numero du vol : "; getline(cin,n);
        if (trouverVol(n) == -1) { cout << "Vol introuvable.\n"; return; }
        for (const auto& r : reservations)
            if (r.getIdPassager()==id && r.getNumeroVol()==n && r.active()) {
                cout << "Refus : double reservation.\n"; return;
            }
        if (placesRestantes(n) <= 0) {
            cout << "Refus : vol complet.\n"; return;
        }
        cout << "Date de reservation : "; getline(cin,dt);
        reservations.emplace_back(idR++,id,n,dt);
        cout << "Reservation creee. Numero : " << idR-1
             << ". Elle est en attente de paiement.\n";
    }

    void payer() {
        int id;
        cout << "Numero de reservation : ";
        if (!(cin>>id)) { cin.clear(); viderLigne(); return; }
        viderLigne();
        int ir = trouverReservation(id);
        if (ir == -1) { cout << "Reservation introuvable.\n"; return; }
        if (!reservations[ir].active()) { cout << "Reservation annulee.\n"; return; }
        if (trouverPaiement(id) != -1) { cout << "Deja payee.\n"; return; }
        int iv = trouverVol(reservations[ir].getNumeroVol());
        if (iv == -1) { cout << "Vol introuvable.\n"; return; }
        string dt;
        cout << "Date du paiement : "; getline(cin,dt);
        paiements.emplace_back(idPay++,id,vols[iv].getPrix(),dt);
        reservations[ir].confirmer();
        cout << "Paiement enregistre. Reservation confirmee.\n";
    }

    void annuler() {
        int id;
        cout << "Numero de reservation : ";
        if (!(cin>>id)) { cin.clear(); viderLigne(); return; }
        viderLigne();
        int ir = trouverReservation(id);
        if (ir == -1) { cout << "Reservation introuvable.\n"; return; }
        if (!reservations[ir].active()) { cout << "Deja annulee.\n"; return; }
        reservations[ir].annuler();
        int ip = trouverPaiement(id);
```

```
        if (ip != -1 && paiements[ip].getStatut()=="Paye") {
            paiements[ip].rembourser();
            cout << "Reservation annulee ; paiement marque rembourse.\n";
        } else cout << "Reservation annulee ; aucun paiement a rembourser.\n";
        cout << "La place est liberee.\n";
    }

    void consulterReservations() const {
        if (reservations.empty()) cout << "Aucune reservation.\n";
        for (const auto& r : reservations) r.afficher();
    }

    void detailsReservation() const {
        int id;
        cout << "Numero de reservation : ";
        if (!(cin>>id)) { cin.clear(); viderLigne(); return; }
        viderLigne();
        int ir = trouverReservation(id);
        if (ir == -1) { cout << "Reservation introuvable.\n"; return; }
        reservations[ir].afficher();
        int ip = trouverPaiement(id);
        if (ip == -1) cout << "Aucun paiement.\n";
        else paiements[ip].afficher();
    }

    void consulterPaiements() const {
        if (paiements.empty()) cout << "Aucun paiement.\n";
        for (const auto& p : paiements) p.afficher();
    }

    void menu() {
        int choix;
        do {
            cout << "\n===== AGENCE DE VOYAGES =====\n"
                 << "1. Enregistrer un passager\n"
                 << "2. Ajouter un vol\n"
                 << "3. Afficher les vols disponibles\n"
                 << "4. Rechercher un vol\n"
                 << "5. Effectuer une reservation\n"
                 << "6. Enregistrer un paiement\n"
                 << "7. Annuler une reservation\n"
                 << "8. Consulter les reservations\n"
                 << "9. Afficher les details d'une reservation\n"
                 << "10. Quitter\n"
                 << "11. Afficher les passagers\n"
                 << "12. Consulter les paiements\nVotre choix : ";
            if (!(cin>>choix)) { cin.clear(); viderLigne(); choix=0; }
            else viderLigne();
            switch (choix) {
                case 1: ajouterPassager(); break;
                case 2: ajouterVol(); break;
                case 3: afficherVols(); break;
                case 4: rechercherVol(); break;
                case 5: reserver(); break;
                case 6: payer(); break;
                case 7: annuler(); break;
                case 8: consulterReservations(); break;
                case 9: detailsReservation(); break;
                case 10: cout << "Fin du programme.\n"; break;
                case 11: afficherPassagers(); break;
                case 12: consulterPaiements(); break;
                default: if (choix!=10) cout << "Choix invalide.\n";
            }
        } while (choix != 10);
    }
};

int main() {
    AgenceVoyage agence;
    agence.menu();
    return 0;
}
```

7. Explication du code

Passager : stocke les informations du client. L'identifiant est créé automatiquement et sert à retrouver le passager.

Vol : stocke le numéro, le trajet, la date, la capacité et le prix. Les méthodes d'accès permettent de consulter ces informations sans rendre les attributs publics.

Reservation : conserve l'identifiant du passager et le numéro du vol. La méthode active() permet de savoir si la réservation compte encore dans la capacité du vol.

Paiement : conserve le montant et l'identifiant de la réservation payée. Le statut passe de « Paye » à « Rembourse » lors d'une annulation payée.

AgenceVoyage : centralise les vecteurs et les opérations. Les fonctions trouverPassager(), trouverVol() et trouverReservation() recherchent les éléments. placesRestantes() calcule la capacité encore disponible.

8. Menu et opérations

Le menu propose les dix opérations demandées dans le sujet. Deux opérations supplémentaires permettent d'afficher les passagers et les paiements pour faciliter les vérifications.

9. Tests de validation

Test	Action	Résultat attendu
1	Réserver avec un identifiant de passager inconnu.	Refus : passager non enregistré.
2	Saisir un numéro de vol inexistant.	Refus : vol introuvable.
3	Réserver deux fois le même vol avec le même passager.	La deuxième réservation est refusée.
4	Réserver un vol sans place disponible.	Refus : vol complet.
5	Payer une réservation en attente.	Paiement enregistré et réservation confirmée.
6	Annuler une réservation payée.	Réservation annulée, paiement remboursé, place libérée.
7	Tenter de payer une réservation annulée.	Paiement refusé.

10. Compiler et exécuter

Enregistre le code dans main.cpp. Avec un compilateur g++, utilise la commande :
g++ -std=c++17 main.cpp -o agence
Puis lance le programme avec : ./agence. Sur un site de compilation C++, colle le code dans l'éditeur, choisis le standard C++17 et clique sur Run/Exécuter.

11. Conclusion et limites

Le projet applique les notions de programmation orientée objet : classes, objets, encapsulation, constructeurs, méthodes et vecteurs. Le diagramme UML représente la structure statique ; le programme C++ met en œuvre les opérations et les règles de gestion.

Les données ne sont pas persistantes : elles disparaissent lorsque le programme se ferme. Pour un système réel, il faudrait ajouter une base de données ou des fichiers de sauvegarde, une authentification, des validations plus poussées des dates et une intégration réelle des paiements et remboursements.